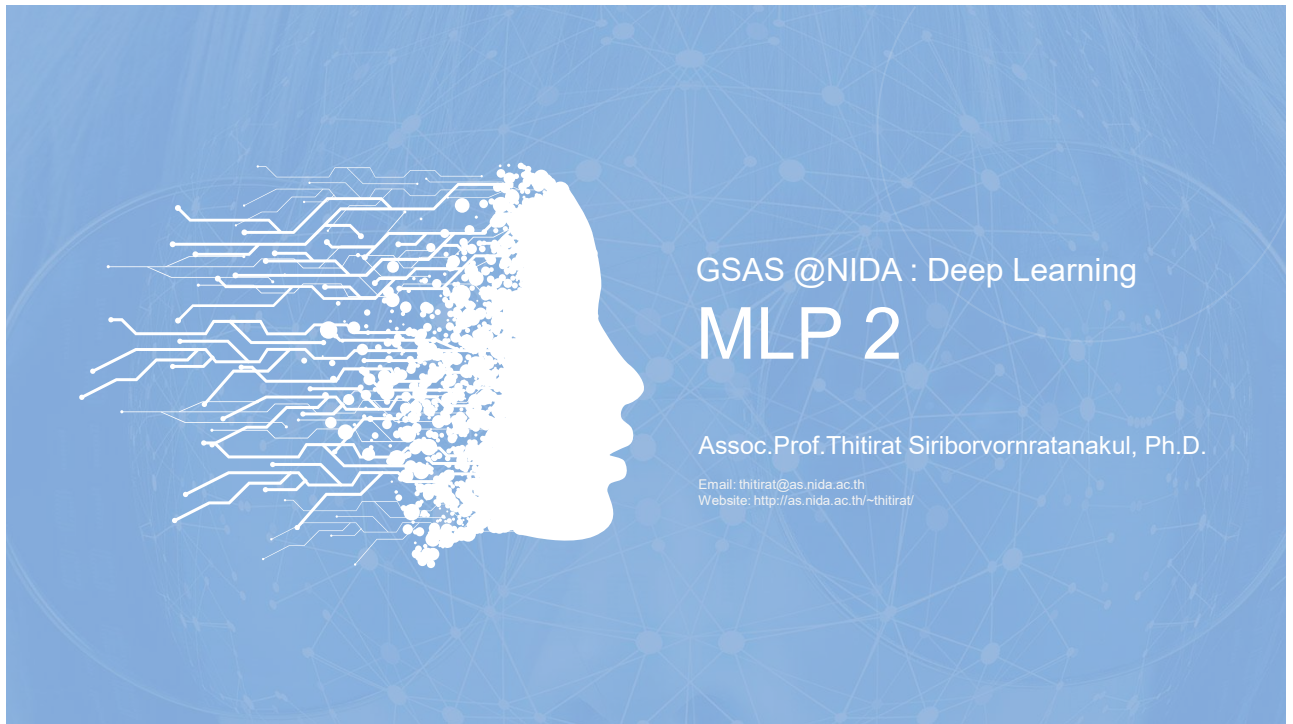




1

OUTLINE

01 Problem 1

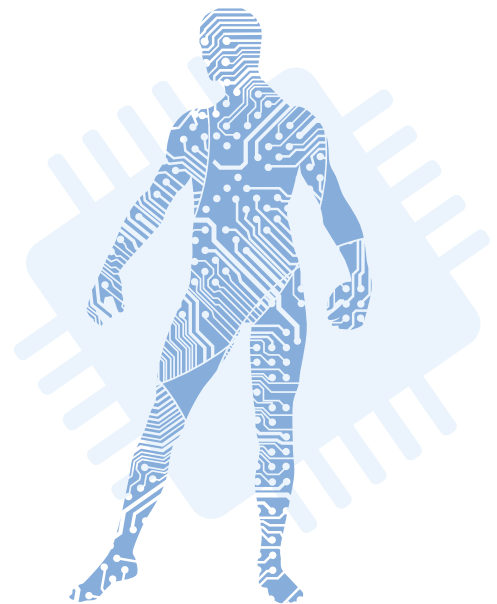
Unstable gradients in deep neural networks and how to deal with it

02 Problem 2

Overfitting in deep neural networks and how to solve it

03 Coding

Combine everything and implement a program with Keras



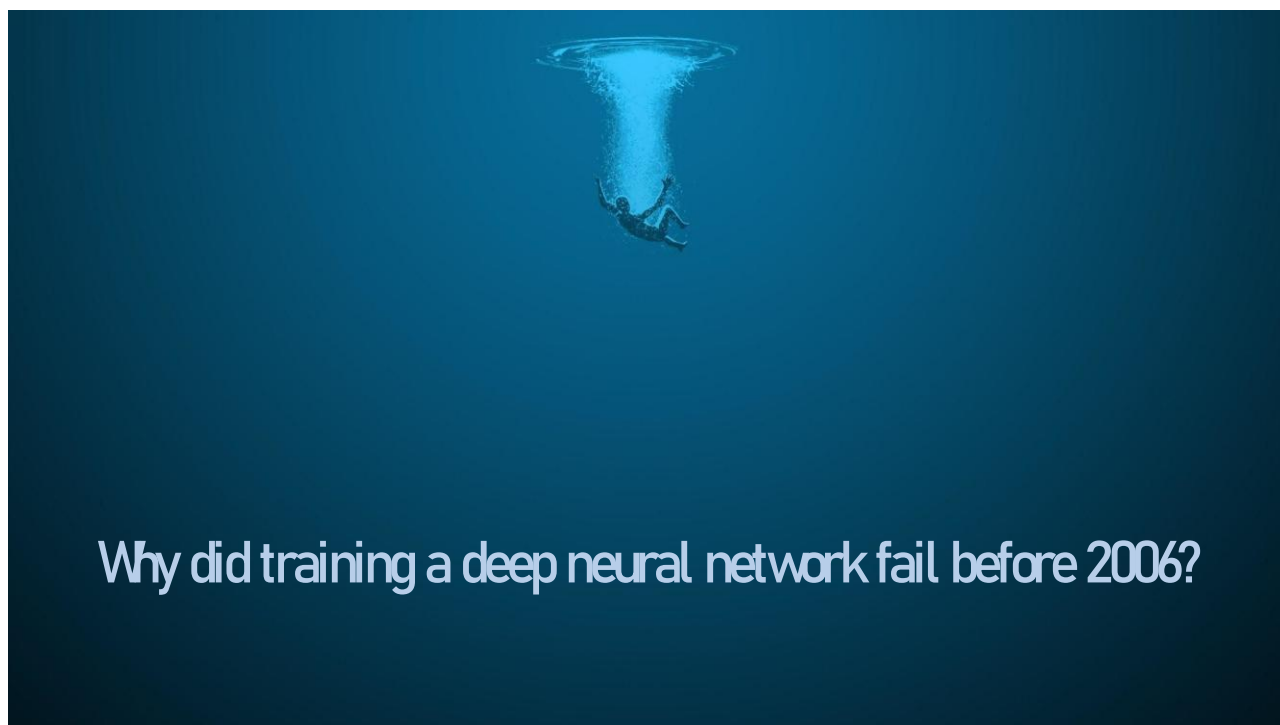
2



Problem 1

Unstable gradients in deep neural networks
and how to deal with it

3



Why did training a deep neural network fail before 2006?

4

A Gradient Vanishing

- In the early 2000s:
 - Gradients get smaller and smaller as the algorithm progresses down to the lower layers.
 - As a result, the Gradient Descent update leaves the lower layers' connection weights virtually unchanged, and training never converges to a good solution.
- **Gradient vanishing** was one of the reasons deep neural networks were mostly abandoned in the early 2000s.



5

A Problem of unstable gradients

- **Gradient vanishing:**
 - Slow training progress
 - Low weight updates
 - Dead neurons: Neurons in some layers have very small or zero outputs, causing them to become inactive.
- **Gradient exploding:** Large updates to weights (= large gradients) during training that cause a numerical overflow or underflow.
 - Large weight updates
 - Null, NaN, or Inf values in weights
 - Oscillating performance: The model's accuracy fluctuates over time, indicating instability.
- How to detect unstable gradients?
 - Monitor the magnitude of gradients (whether they are too small or too large) during the training process using a real-time visualization tool (e.g., TensorBoard)

6

A Tackle unstable gradients

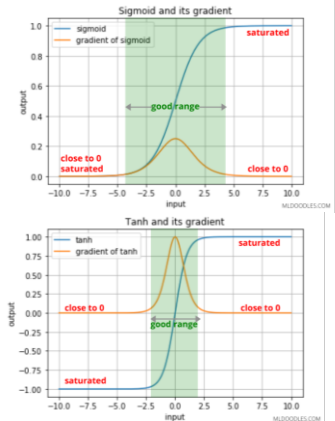
- The work of Glorot and Bengio (2010) found a few suspects to the unstable gradient when training a deep neural network.

- Nonsaturating activation functions:**

- Other activation functions (e.g., ReLU) behave much better in deep neural networks than sigmoid. This is because ReLU does not saturate for positive values.

- Weight initialization:**

- It took over a decade for researchers to realize how important this trick is.
 - Using **Glorot initialization** can speed up training considerably, and it is one of the tricks that led to the success of Deep Learning.



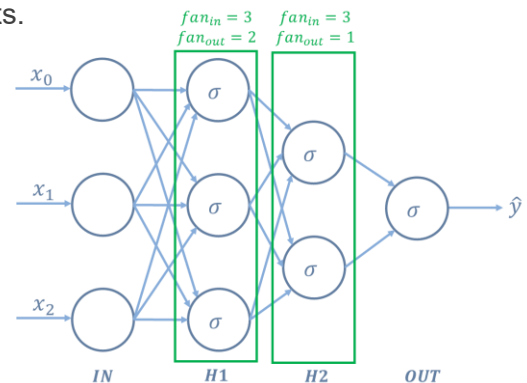
Xavier Glorot and Yoshua Bengio, "Understanding the difficulty of training deep feedforward neural networks," In Proc. of Machine Learning Research, pp.249-256, 2010.

7

A Initialization strategy

- A too-large initialization leads to exploding gradients.
- A too-small initialization leads to vanishing gradients.

- For signal to flow properly in both forward and backward directions without dying out, exploding or saturating, we need the gradients to have equal variance before and after flowing through a layer in the reverse direction.
 - The mean of the activations should be zero.
 - The variance of the activations should stay the same across every layer.

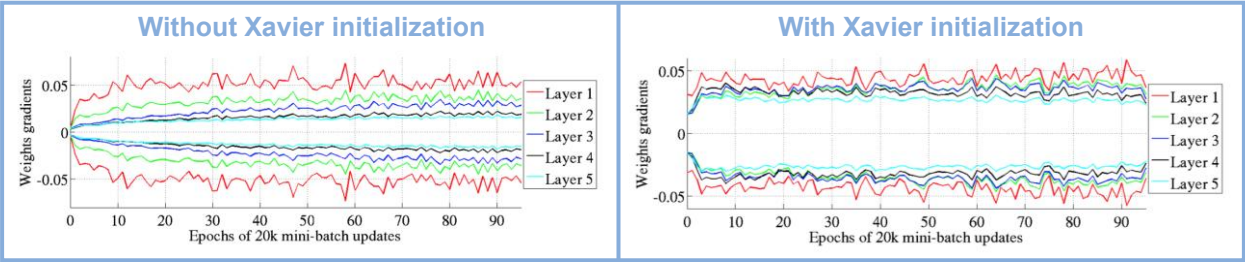


- Unfortunately, it is not possible to guarantee both unless the layer has an equal number of incoming network connections (fan_{in}) and outgoing network connections (fan_{out}).

8

Initialization strategy

- **Xavier initialization** (a.k.a., **Glorot initialization**) proposed a good compromise that has proven to work very well in practice, just setting a layer's weights to values chosen from a random uniform distribution that's bounded between $\left[-\sqrt{\frac{6}{fan_{in}+fan_{out}}} , \sqrt{\frac{6}{fan_{in}+fan_{out}}}\right]$



Xavier Glorot and Yoshua Bengio, "Understanding the difficulty of training deep feedforward neural networks," In Procs. of Machine Learning Research, pp.249-256, 2010.

9

Initialization strategy

- Keras uses **Glorot initialization** with a uniform distribution by default. This can be changed in `kernel_initializer` when creating a new layer.

Initialization strategy	Activation functions	Uniform distribution [-A , A]	Normal distribution (with mean = 0)
Glorot, Xavier (2010)	None, tanh, sigmoid, softmax	$A = \sqrt{\frac{6}{fan_{in} + fan_{out}}}$	$\sigma = \sqrt{\frac{2}{fan_{in} + fan_{out}}}$
He (2015)	ReLU and variants	—	$\sigma = \sqrt{\frac{2}{fan_{in}}}$
LeCun (1998)	SELU	—	$\sigma = \sqrt{\frac{1}{fan_{in}}}$

```
keras.layers.Dense(  
    units,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    lora_rank=None,  
    lora_alpha=None,  
    **kwargs  
)
```



10

A BATCH NORMALIZATION (ICML 2015)

- Using weight initialization and non-saturating activation function can significantly reduce the danger of the vanishing/exploding gradients problems at the beginning of training.
- BUT** it doesn't guarantee that they won't come back during training.
- Why using the **Batch Normalization (BN)** layer? 
 - Make sure that the output stays within a fixed range, the same logic that makes us perform the input normalization
 - Ioffe and Szegedy (2015) proposed several advantages of **BN**:
 - BN** considerably improved all experimental deep learning networks, leading to a huge improvement in the ImageNet classification task.
 - The vanishing gradient problem was strongly reduced to the point that they could use saturating activation functions like tanh and sigmoid.
 - The networks were much less sensitive to the weight initialization.
 - Smoothed the loss landscape which allowed training with larger learning rates, significantly speeding up the learning process
 - BN** acts like a regularizer, reducing the need for other regularization techniques.



Ioffe and Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," In Procs. of ICML, 2015

11

A BATCH NORMALIZATION (ICML 2015)

- For a hidden layer, it is still arguable whether to insert the **BN** layer before or after the activation function.
 - The original paper of Ioffe and Szegedy (2015) placed the **BN** layer before the activation function.
 - To do this in Keras, don't forget to remove the activation function from the hidden layers and add them as separate layers after the **BN** layers.
 - However, some say placing the **BN** layer after the activation function is better.

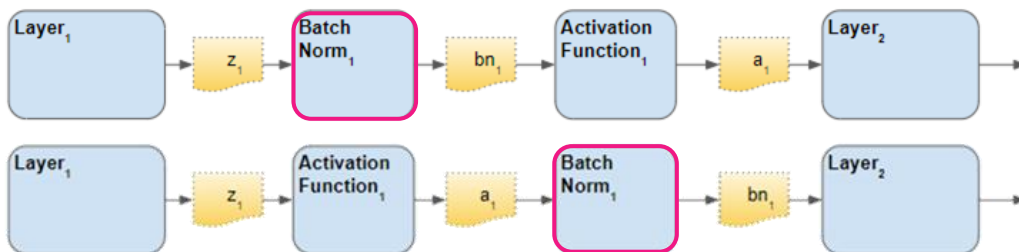


Image credit: (18MAY2021) <https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739>

12

A BATCH NORMALIZATION (ICML 2015)

TRAINING

1.	Mini-batch mean	$\mu_B = \frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} (x_i)$
2.	Mini-batch variance	$(\sigma_B)^2 = \frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} (x_i - \mu_B)^2$
3.	Normalize	$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{(\sigma_B)^2 + \epsilon}}$
4.	Scale and shift	$y_i = \gamma \hat{x}_i + \beta$
5.	Exponential moving average	$\mu_{mov} = \alpha \cdot \mu_{mov} + (1 - \alpha) * \mu_B$ $(\sigma_{mov})^2 = \alpha \cdot (\sigma_{mov})^2 + (1 - \alpha) * (\sigma_B)^2$

During training, it simply calculates this EMA but does not do anything with it.
At the end of training, it simply saves this value as part of the layer's state, for use during the Inference phase.

Batch Norm

Learnable Params: Beta (β), Gamma (γ)
Saved Params: Moving Avg (mean), Moving Avg (Var)

$x_i \rightarrow y_i$

The momentum hyperparameter of BN

13

A BATCH NORMALIZATION (ICML 2015)

- A single **BN** layer has four parameter vectors:
 - Two learnable (= trainable through backprop) parameter vectors called γ and β that refer to scale and shift (offset) parameter vectors.
 - Two non-learnable (= non-trainable) parameter vectors—Mean Moving Average (μ_{mov}) and Variance Moving Average ($(\sigma_{mov})^2$)—which are saved as part of the state of the **BN** layer.

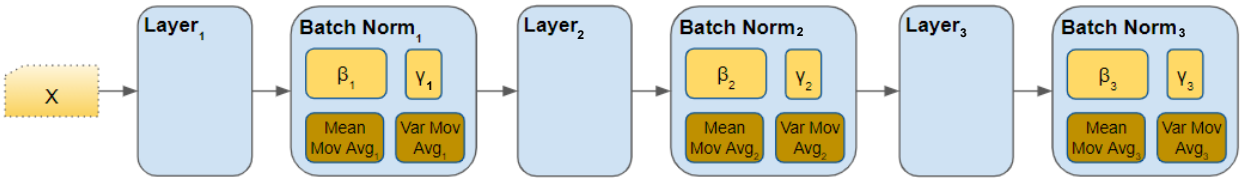


Image credit: (18MAY2021) <https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739>

14

A BATCH NORMALIZATION (ICML 2015)

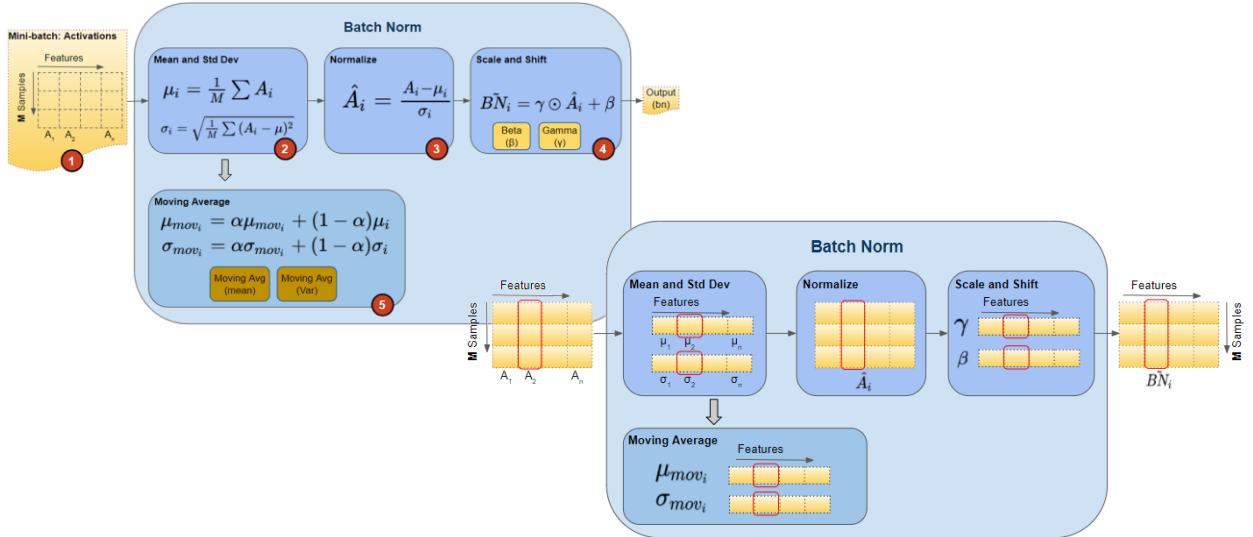


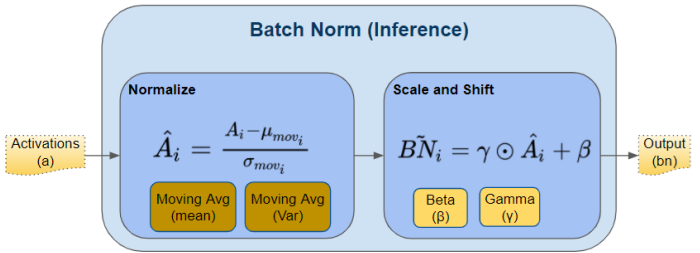
Image credit: (18MAY2021) <https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739>

15

A BATCH NORMALIZATION (ICML 2015)

INFERENCE

- At inference, there may be no batch of samples, or a batch may be too small so that samples may not be independent and identically distributed.
- Most **BN** implementations use the μ_{mov} and $(\sigma_{mov})^2$ values of the **BN** layer (precomputed during training) to replace the batch input means (μ_B) and SDs (σ_B) during inference.



1. Mini-batch mean	$\mu_B = \frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} (x_i)$
2. Mini-batch variance	$(\sigma_B)^2 = \frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} (x_i - \mu_B)^2$
3. Normalize	$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{(\sigma_B)^2 + \epsilon}}$
4. Scale and shift	$y_i = \gamma \hat{x}_i + \beta$
5. Exponential moving average	$\mu_{mov} = \alpha \cdot \mu_{mov} + (1 - \alpha) \cdot \mu_B$ $(\sigma_{mov})^2 = \alpha \cdot (\sigma_{mov})^2 + (1 - \alpha) \cdot (\sigma_B)^2$

Image credit: (18MAY2021) <https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739>

16

K CODE REVIEW

```
keras.layers.BatchNormalization(
    axis=-1,
    momentum=0.99,
    epsilon=0.001,
    center=True,
    scale=True,
    beta_initializer="zeros",
    gamma_initializer="ones",
    moving_mean_initializer="zeros",
    moving_variance_initializer="ones",
    beta_regularizer=None,
    gamma_regularizer=None,
    beta_constraint=None,
    gamma_constraint=None,
    synchronized=False,
    **kwargs
)
```

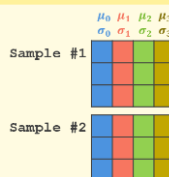
By default, normalize the last axis by using the means and SDs computed across the other axes

- **axis**: Integer, the axis that should be normalized (typically the features axis). For instance, after a `Conv2D` layer with `data_format="channels_first"`, set `axis=1` in `BatchNormalization`.

axis=-1
`input.shape`
`= [ batch_size, features ]`
`= [ 3, 5 ]`



axis=-1
`input.shape`
`= [ batch_size, H, W ]`
`= [ 2, 3, 4 ]`



axis=[1,2]
`input.shape`
`= [ batch_size, H, W ]`
`= [ 2, 3, 4 ]`



- All 12 pixels are treated independently.
- So, there are 12 means and 12 SDs.

A

17

```
keras.layers.Layer(
    activity_regularizer=None,
    trainable=True,
    dtype=None,
    autocast=True,
    name=None,
    **kwargs
)
```

K BatchNormalization layer

During training (i.e. when using `fit()` or when calling the layer/model with the argument `training=True`), the layer normalizes its output using the mean and standard deviation of the current batch of inputs. That is to say, for each channel being normalized, the layer returns $\gamma * (\text{batch} - \text{mean}(\text{batch})) / \sqrt{\text{var}(\text{batch}) + \epsilon} + \beta$, where:

- **epsilon** is small constant (configurable as part of the constructor arguments)
- **gamma** is a learned scaling factor (initialized as 1), which can be disabled by passing `scale=False` to the constructor.
- **beta** is a learned offset factor (initialized as 0), which can be disabled by passing `center=False` to the constructor.

During inference (i.e. when using `evaluate()` or `predict()` or when calling the layer/model with the argument `training=False` (which is the default)), the layer normalizes its output using a moving average of the mean and standard deviation of the batches it has seen during training. That is to say, it returns $\gamma * (\text{batch} - \text{self.moving_mean}) / \sqrt{\text{self.moving_var} + \epsilon} + \beta$.

`self.moving_mean` and `self.moving_var` are non-trainable variables that are updated each time the layer is called in training mode, as such:

- `moving_mean = moving_mean * momentum + mean(batch) * (1 - momentum)`
- `moving_var = moving_var * momentum + var(batch) * (1 - momentum)`

As such, the layer will only normalize its inputs during inference *after having been trained on data that has similar statistics as the inference data*.

A

18

AI BATCH NORMALIZATION (ICML 2015)

- **BN** has become one of the most-used layers in deep neural networks, to the point that it is often omitted in the diagrams, as it is assumed that **BN** is added after every layer.
- Why not using the **Batch Normalization (BN)** layer? 🗨️
 - Run time penalty (slower predictions due to the extra computation)
 - This can be solved by fusing the **BN** layer with the previous layer, after training (TFLite's optimizer does this automatically).
 - **BN** adds memory overhead and is a common source of implementation errors.
 - **BN**'s behavior is strongly dependent on the batch size, performing poorly when the per device batch size is too small or too large.
 - **BN** behaves differently during training and inference.
 - Intra-batch information leakage breaks the independence between training samples in the mini-batch, resulting in many implementation errors especially in distributed training.
 - It's difficult to replicate batch normalized models trained on different hardware.
 - **BN** implementation is said to be different on each deep learning framework.
 - The mystery regarding the success of **BN** in deep networks is still unclear.



Google Deepmind, "High-Performance Large-Scale Image Recognition Without Normalization," In arXiv, 2021 <https://arxiv.org/abs/2102.06171>.
Google Deepmind, "Characterizing signal propagation to close the performance gap in unnormalized ResNets," In Procs. of ICLR, 2021 <https://arxiv.org/abs/2101.08692>

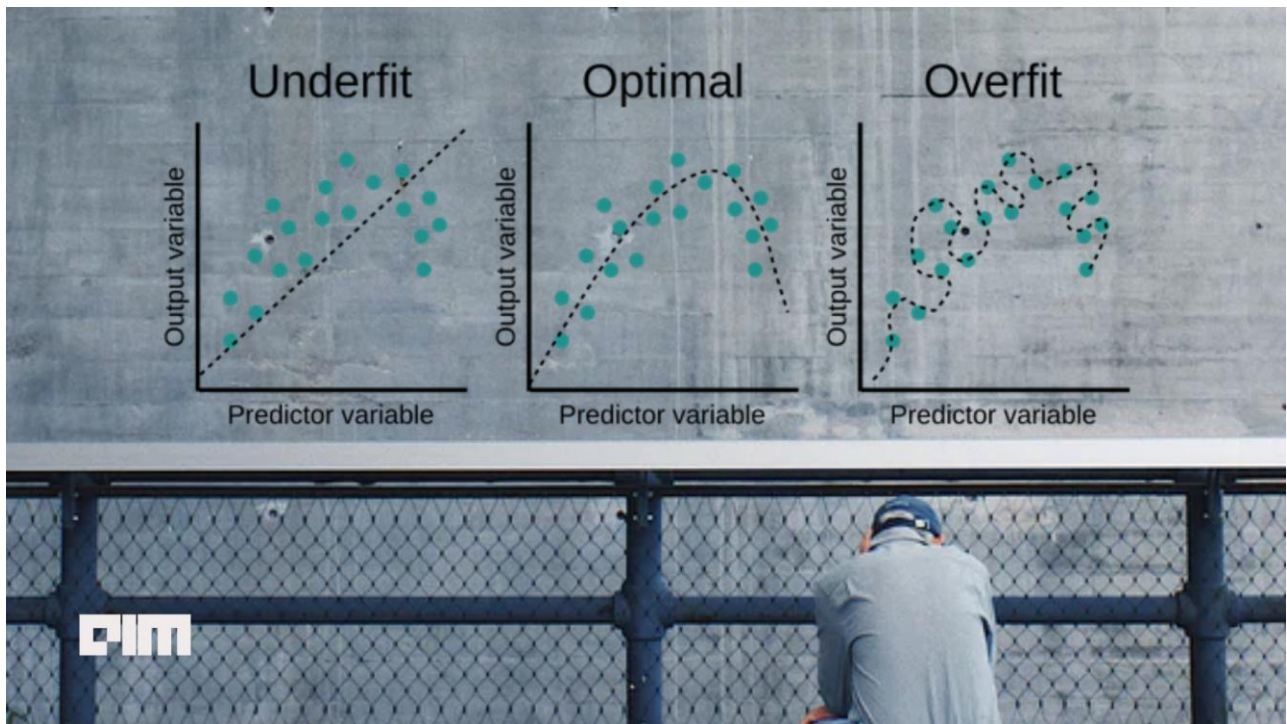
19



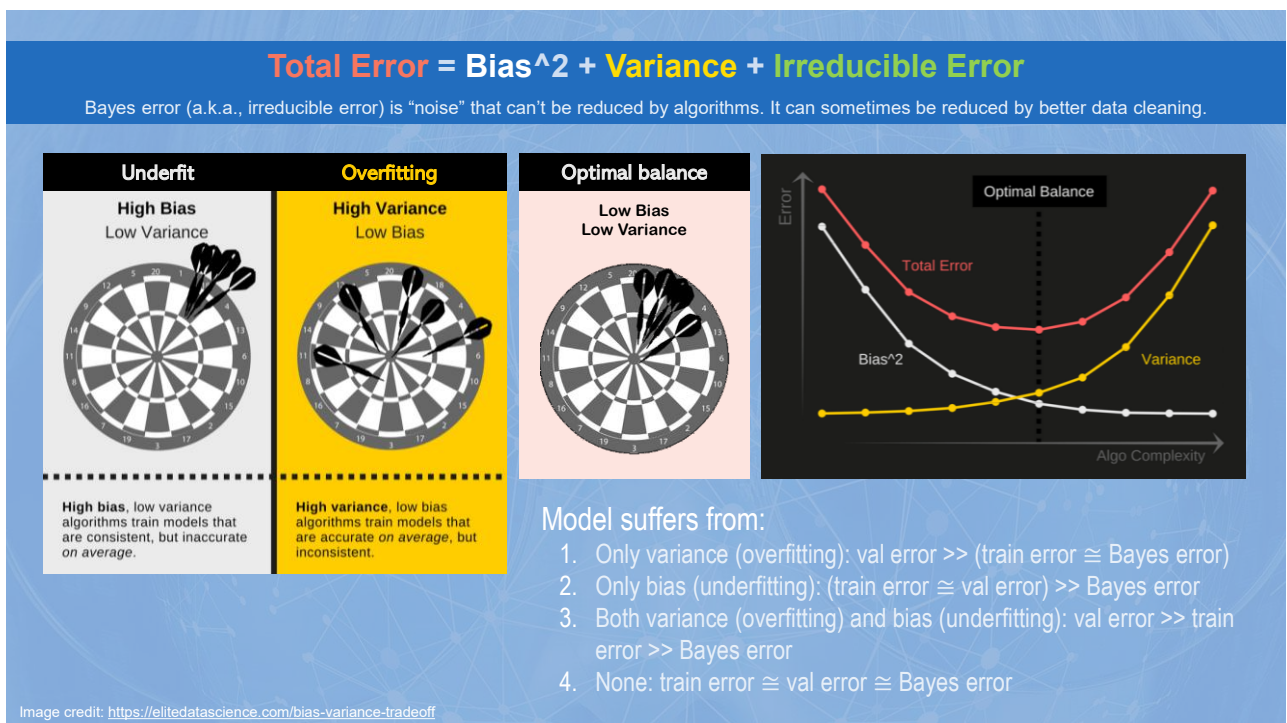
Problem 2

Overfitting in deep neural networks and how to solve it

20



21

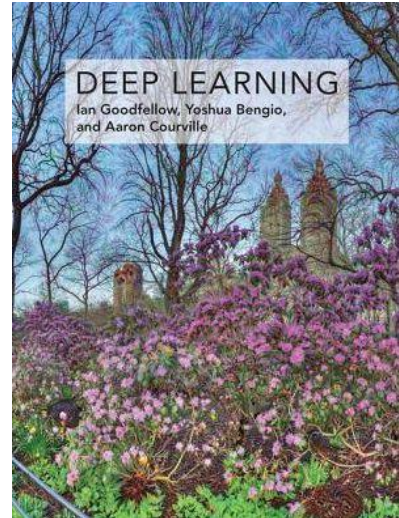


22

A REGULARIZATION

“**Regularization** is any modification we make to a learning algorithm that is intended to **reduce its generalization error** but not its training error.”

↓
Reduce overfit



23

A Regularization in Deep NN

- Batch Normalization (2015)

- Early stopping

- L1 and L2 regularization

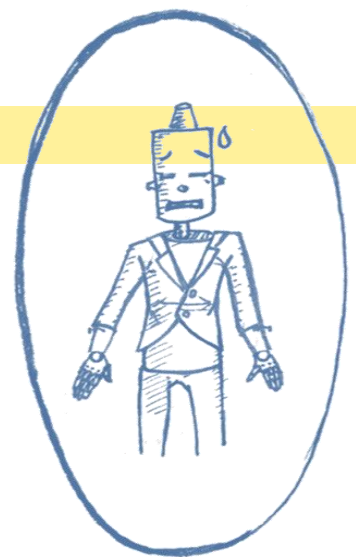
- Dropout (2012)

- Data augmentation

- ...

} Lower the complexity of the network during training

} Increase the training dataset



24

A Early Stopping

- **Double descent phenomenon** occurs in CNNs, ResNets, and Transformers.
 - Performance first improves, then gets worse, and then improves again with increasing model size, data size, or training time. $\Rightarrow$ This implies that, finally, big models will converge. As long as we, in practice, have enough computational resources to train them.
 - This effect is often avoided through careful regularization. While this behavior appears to be fairly universal, we don't yet fully understand why it happens, and view further study of this phenomenon as an important research direction.

```
keras.callbacks.callbacks.EarlyStopping
(monitor='val_loss', min_delta=0,
patience=0, verbose=0, mode='auto',
baseline=None,
restore_best_weights=False,
start_from_epoch=0)
```



[5DEC2019] <https://openai.com/blog/deep-double-descent/>
[10DEC2018] <https://machinelearningmastery.com/how-to-stop-training-deep-neural-networks-at-the-right-time-using-early-stopping/>

25

A Regularization in Deep NN

• Batch Normalization (2015)

• Early stopping

• L1 and L2 regularization

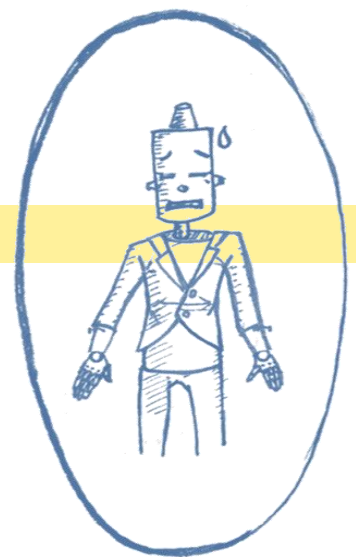
• Dropout (2012)

• Data augmentation

• ...

Lower the
complexity of the
network during
training

Increase the
training dataset



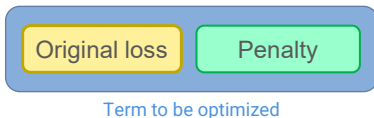
26

A L1 & L2 REGULARIZATION

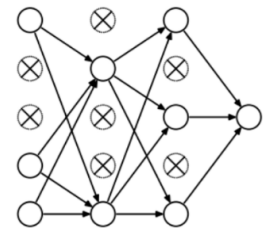
- **L1** and **L2** are the most common types of regularization that update the general cost/loss function by adding another term known as **the regularization term (a.k.a., the penalty term)**.

$$\underbrace{-\sum_{i=1}^{\text{output size}} [y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log(1 - \hat{y}_i)]}_{\text{A loss function (e.g., BCE loss)}} + \underbrace{\text{RegularizationTerm}}_{\text{The penalty term}}$$

- What are the consequences of having higher loss?



LOSS ↑ w_i ↓



27

A L1 & L2 RE

L2 Regularization and Weight Decay are only the same thing for vanilla SGD, but as soon as we add momentum, or use a more sophisticated optimizer like Adam, L2 regularization and weight decay become different.

- **L2 Regularization** (a.k.a., **Weight decay**)
 - The most common type of all regularization techniques
 - Encourage the weight values towards zero (but not exactly zero)
 - The regularization term is defined as the Euclidean Norm (or L2 norm) of the weight matrices.
- **L1 Regularization** (a.k.a., **Lasso regression**)
 - Encourage the weight values to be zero, resulting in a sparse model
 - The regularization term is the sum of the absolute values of the weight parameters in a weight matrix.
- λ is sometimes called as **the regularization rate/parameter** and is an additional hyperparameter to determine how much we regularize our model.

$$Loss = Error(y, \hat{y})$$

Loss function with no regularisation

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$$

Loss function with L1 regularisation

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N w_i^2$$

Loss function with L2 regularisation

28

K CODE REVIEW

```
keras.layers.Dense(  
    units,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    lora_rank=None,  
    lora_alpha=None,  
    **kwargs  
)
```

"l1" # the default lambda is l1=0.01
keras.regularizers.l1(1e-5)

"l2" # the default lambda is l2=0.01
keras.regularizers.l2(1e-4)

keras.regularizers.l1_l2(l1=1e-5, l2=1e-4)

A

29

A L1 & L2 REGULARIZATION

```
keras.layers.Dense(  
    units,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    lora_rank=None,  
    lora_alpha=None,  
    **kwargs  
)
```

- **kernel_regularizer**: Apply regularization only to the weights of the layer
 - Penalize large weights. Try to reduce weights (excluding bias).
- **bias_regularizer**: Apply regularization only to the biases of the layer
 - Penalize large biases. Try to reduce bias.
 - However, regularizing bias values may not change the results significantly.
- **activity_regularizer**: Apply regularization only to the output of the layer
 - Penalize large outputs. Thus, it will reduce the weights and adjust the bias so the layer's output is the smallest.
 - A use case to get sparse outputs/representations from AutoEncoder <https://stackoverflow.com/questions/44495698/keras-difference-between-kernel-and-activity-regularizers>

30

Regularization in Deep NN

- ~~Batch Normalization (2015)~~

- Early stopping

- L1 and L2 regularization

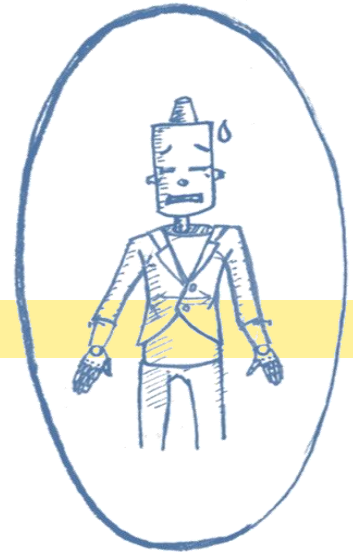
- Dropout (2012)

- Data augmentation

- ...

Lower the
complexity of the
network during
training

Increase the
training dataset



31

DROPOUT (2012)

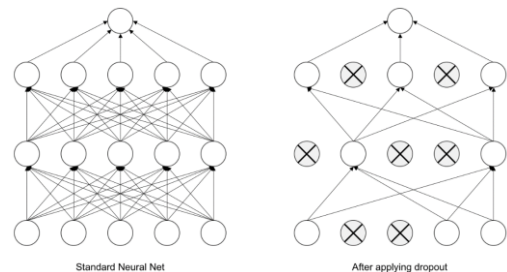
- **Dropout** by Hinton et al. (2012) is one of the most popular regularization techniques for deep neural networks.

- **During training:**

- At every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily “dropped out.”
- p is a new hyperparameter called **the dropout rate**, and typically set between 10% to 50% (20-30% in RNN and 40-50% in CNN).

- Why does **dropout** help reduce overfit?

- At each training step, a smaller network is trained.
- A network can't rely on any one feature (= can't bet a large value on any one weight) and has to spread out weights, resulting in weight shrinking (similar to the L2 regularization).

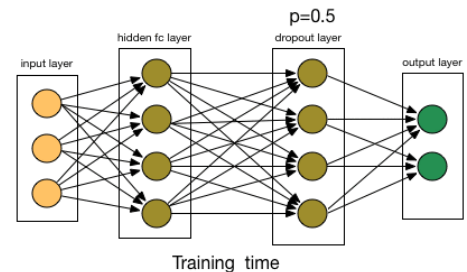


Hinton et al., "Improving neural networks by preventing co-adaptation of feature detectors," CoRR, 2012 <https://arxiv.org/abs/1207.0580>

32

A DROPOUT (2012)

- **Dropout** also helps explore many sub-networks and can be thought of as an ensemble technique. Ensemble models usually perform better than a single model as they capture more randomness.
- Where to apply **dropout**?
 - Dropout can be applied to any layer but it is uncommon to apply dropout to input (and output) layers.
 - We may apply dropout with a high dropout rate to a layer with a large number of trainable parameters (as it has a high risk of overfitting), and a low dropout rate otherwise.
 - In the practice of computer vision tasks, we usually apply dropout only to the neurons in the top one to three (FC) layers (excluding the output layer).
- **During inference:** To make sure that the sum over all inputs is unchanged during training (when **dropout** is active) and during inference (when **dropout** is inactive):
 - During inference, multiply each input connection weight (of the layer after dropout) by the keep probability ($1 - p$), **OR**
 - During training, divide each neuron's output (of the layer before dropout) by the keep probability



33

K CODE REVIEW

```
keras.layers.Dropout(rate, noise_shape=None, seed=None, **kwargs)
```

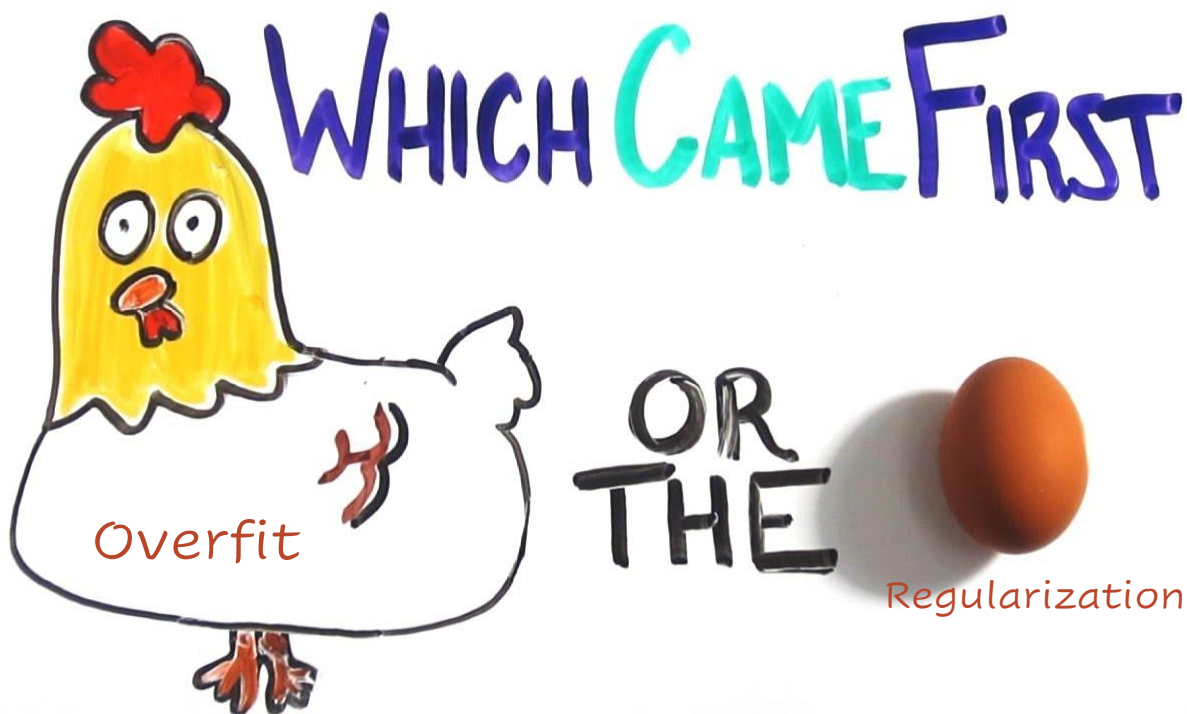
```
from keras.layers.core import Dropout

model = Sequential([
    Dense(output_dim=hidden1_num_units, input_dim=input_num_units, activation='relu'),
    Dropout(0.25), # rate is between 0.0-1.0 to represent fraction of the input units to drop
    Dense(output_dim=output_num_units, input_dim=hidden5_num_units, activation='softmax'),
])
```

- Since dropout is only active during training, comparing the training loss and the validation loss can be misleading.
 - In particular, a model may be overfitting the training set and yet have similar training and validation losses.
 - So make sure to evaluate the training loss without dropout (e.g., after training).



34



35



36



Coding

Combine everything and implement a program with Keras

37

EX 1: MLP for Tabular Data

- Keras's built-in (structure) datasets:
 - Boston Housing price dataset https://keras.io/api/datasets/boston_housing/
- **Tips & Tricks:**
 - In practice, K -fold cross-validation is not frequently used in deep learning due to the very expensive cost of training K different deep learning models (This issue was mentioned in the famous YouTube class of Stanford @34:00 <https://www.youtube.com/watch?v=OoUX-nOEjG0&list=PL3FW7Lu3i5JvHM8ljYj-zLfQRF3EO8sYv&index=2>).
 - The model obtained at the last epoch may not be the best model that yields the minimum validation loss. In order to tell Keras to save the best model, use this callback https://keras.io/api/callbacks/model_checkpoint/ .
 - For long (and serious) training, try using interactive visualization tools to log and visualize training results (e.g., train loss, val loss, image) in real time.



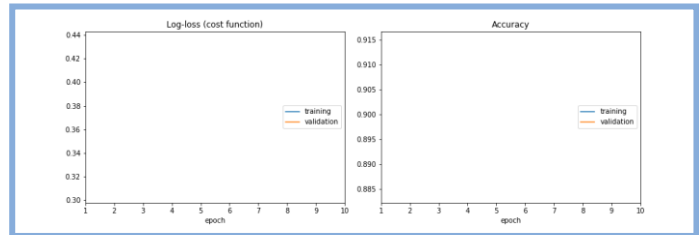
38

Interactive Visualization

• Livelossplot:

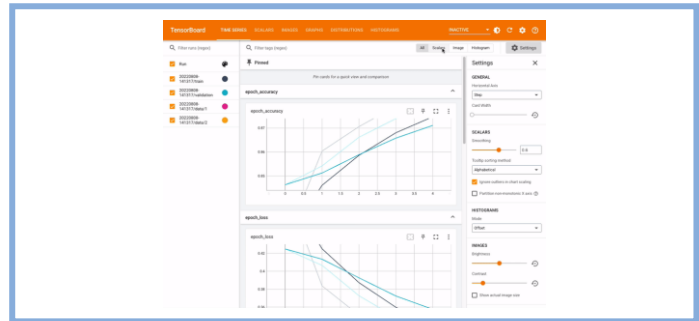
<https://github.com/stared/livelossplot>

- Simple installation and usage
- Jupyter notebook compatibility



• TensorBoard:

- Popular visualization and logging tools in deep learning for both TensorFlow and PyTorch users
- Can log many types of information
- Open source, run locally



39

Interactive Visualization

• Weights & Biases (WandB): <https://wandb.ai/site>

- Compared with TensorBoard:
 - TensorBoard is an open-source tool that runs locally so it's difficult to track everything when working on different machines.
 - WandB offers managed services that can be deployed on-premises but also run in the cloud.
 - WandB provides wider functionality than TensorBoard, covering experiment tracking, dataset versioning, and model management.
 - WandB has a lot of features that enable team collaboration, something that's missing in TensorBoard.
 - Compared to WandB, TensorBoard doesn't scale well with more experiments.
 - More in <https://neptune.ai/vs/wandb-tensorboard>



40

AI Homework

41

Structured Data: Traditional



vs



- **Objective:** Which one is better for structured data, traditional ML or MLP?
- Compare a self-designed MLP network with one or more traditional machine learning techniques
 - Choose at least one tabular dataset to share among group members; **no duplication with those already used in the class; no duplication between groups; Keras's built-in datasets are not allowed.**
 - Experiment with the hyperparameter tuning by your own self, comparing several combinations of hyperparameters and finding the best one
 - Conclude implementation detail, comparative results (both technical and design aspects), and recommendation about using MLP vs. using traditional ML for structure data



42

Tabular Deep Learning

Revisiting Deep Learning Models for Tabular Data

Yury Gorishniy^{†‡} Ivan Rubachev^{†*} Valentin Khulkov[†] Artem Babenko^{†*}

[†] Yandex, Russia

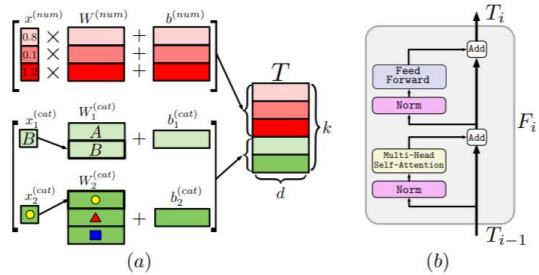
[‡] Moscow Institute of Physics and Technology, Russia

^{*} National Research University Higher School of Economics, Russia

Abstract

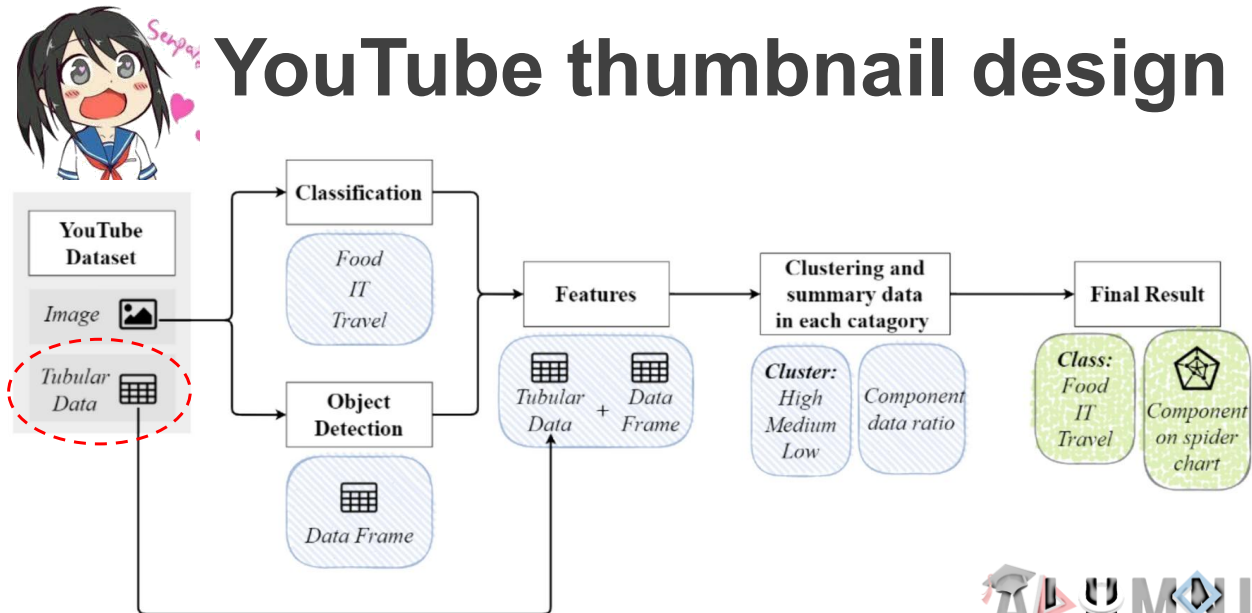
The existing literature on deep learning for tabular data proposes a wide range of novel architectures and reports competitive results on various datasets. However, the proposed models are usually not properly compared to each other and existing works often use different benchmarks and experiment protocols. As a result, it is unclear for both researchers and practitioners what models perform best. Additionally, the field still lacks effective baselines, that is, the easy-to-use models that provide competitive performance across different problems.

In this work, we perform an overview of the main families of DL architectures for tabular data and raise the bar of baselines in tabular DL by identifying two simple and powerful deep architectures. The first one is a ResNet-like architecture which turns out to be a strong baseline that is often missing in prior works. The second model is our simple adaptation of the Transformer architecture for tabular data, which outperforms other solutions on most tasks. Both models are compared to many existing architectures on a diverse set of tasks under the same training and tuning protocols. We also compare the best DL models with Gradient Boosted Decision Trees and conclude that there is still no universally superior solution. The source code is available at <https://github.com/yandex-research/rtdl>.



"Revisiting Deep Learning Models for Tabular Data", In Procs. of NeurIPS, 2021 <https://arxiv.org/abs/2106.11959>

43



"YouTube thumbnail design recommendation systems using image-tabular multimodal data for Thai's YouTube thumbnail," Social Network Analysis and Mining, 2024. <https://link.springer.com/article/10.1007/s13278-024-01317-7>

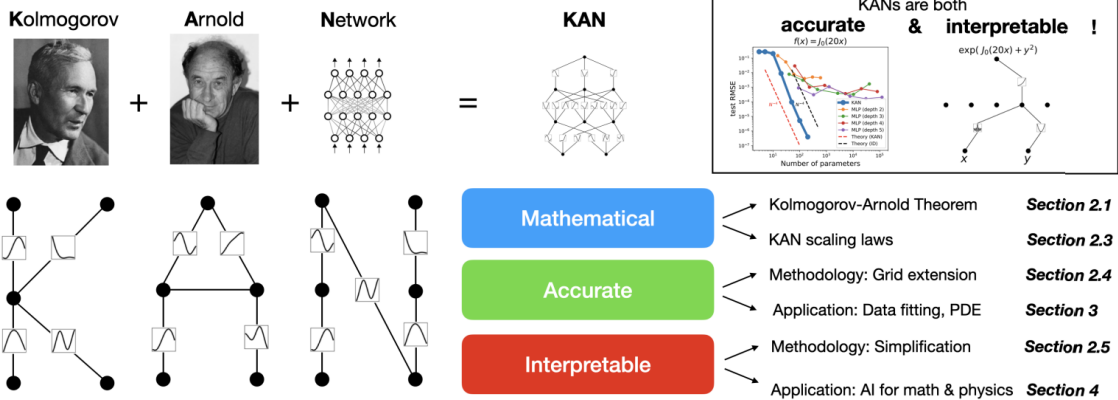
44

AI ETC.

45

Kolmogorov-Arnold Network (arxiv 2024)

- **KANs** are promising alternatives to MLPs.
- **KANs** have strong mathematical foundations just like MLPs.



"KAN: Kolmogorov-Arnold Networks," arxiv30APR2024 <https://arxiv.org/abs/2404.19756> , <https://github.com/KindXiaoming/pykan> , <https://kindxiaoming.github.io/pykan/>

46

Kolmogorov-Arnold Representation Theorem

Kolmogorov-Arnold Representation Theorem (Superposition theorem) established that if f is a multivariate continuous function, then f can be written as a finite composition of continuous functions of a single variable and the binary operation of addition:

$$f(x_1, x_2, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

In other words, this theorem states that any continuous function of n variables can be represented as a composition of $2n+1$ univariate functions.



47

Kolmogorov-Arnold Representation Theorem

# Bedrooms	Area (Sq Ft)	Age	Price
2	18	25	1000
1	6	4	500
2	12	35	1200

Instead of writing in the "multi-variate" form:

$$\text{Price} = F(\text{Bedrooms}, \text{Area}, \text{Age})$$

We can write:

$$\text{Price} = f(\phi_B(\text{Bedrooms}) + \phi_{Sq}(\text{Area}) + \phi_A(\text{Age}))$$

Or more generally:

$$\begin{aligned} \text{Price} = & f_1(\phi_{B1}(\text{Bedrooms}) + \phi_{Sq1}(\text{Area}) + \phi_{A1}(\text{Age})) + \\ & f_2(\phi_{B2}(\text{Bedrooms}) + \phi_{Sq2}(\text{Area}) + \phi_{A2}(\text{Age})) + \\ & f_3(\phi_{B3}(\text{Bedrooms}) + \phi_{Sq3}(\text{Area}) + \phi_{A3}(\text{Age})) + \\ & \dots \\ & f_m(\phi_{Bm}(\text{Bedrooms}) + \phi_{Sqm}(\text{Area}) + \phi_{Am}(\text{Age})) \end{aligned}$$

Hope this formula makes more sense now!

$$Y = \sum_{p=1}^m f_p \left(\sum_{q=1}^n \phi_{pq}(x_q) \right)$$



Image source: 11MAY2024 https://www.youtube.com/watch?v=7zpz_AlFW2w

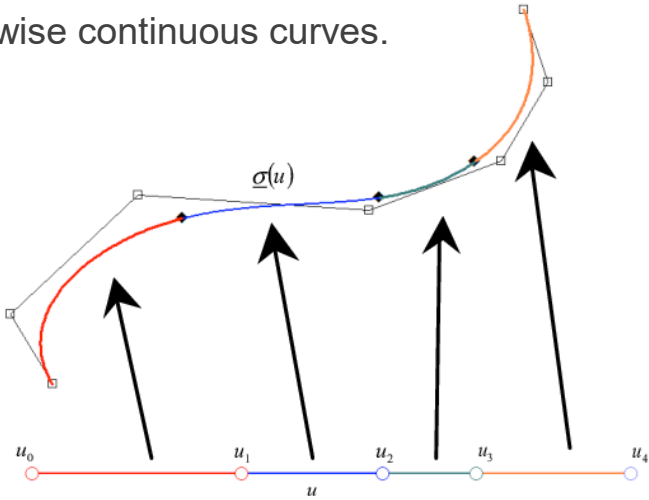
48

B-splines

B-splines are the union of piecewise continuous curves.

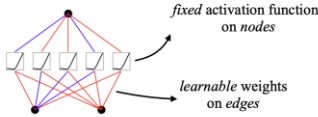
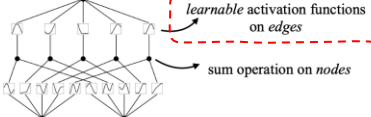
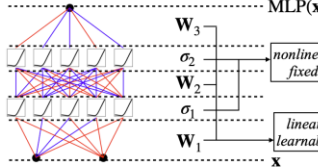
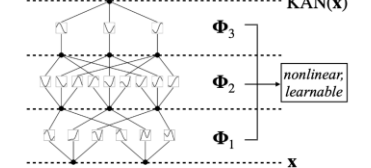
Example of piecewise equations for a cubic B-spline function

$$b^3: \mathbb{R} \rightarrow \mathbb{R}, \quad x \mapsto \left\{ \begin{array}{ll} 0 & \text{für } x \leq -2 \\ \frac{1}{6}x^3 + x^2 + 2x + \frac{4}{3} & \text{für } -2 < x \leq -1 \\ -\frac{1}{2}x^3 - x^2 + \frac{2}{3} & \text{für } -1 < x \leq 0 \\ \frac{1}{2}x^3 - x^2 + \frac{2}{3} & \text{für } 0 < x \leq 1 \\ -\frac{1}{6}x^3 + x^2 - 2x + \frac{4}{3} & \text{für } 1 < x \leq 2 \\ 0 & \text{für } x > 2 \end{array} \right\}$$



49

Kolmogorov-Arnold Network (arxiv 2024)

Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(c)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c)  $\mathbf{W}_3$ σ_2 $\mathbf{W}_2$ σ_1 $\mathbf{W}_1$ $\mathbf{x}$ nonlinear, fixed linear, learnable	(d)  Φ_3 Φ_2 Φ_1 $\mathbf{x}$ nonlinear, learnable

"KAN: Kolmogorov-Arnold Networks," arxiv 30APR2024 <https://arxiv.org/abs/2404.19756> , <https://github.com/KindXiaoming/pykan> , <https://kindxiaoming.github.io/pykan/>

50

Kolmogorov-Arnold Network (arxiv 2024)

- **KANs** use learnable activation functions on edges, entirely eliminating linear weight matrices and replacing them with learnable 1D spline functions. The nodes in a **KAN** simply sum incoming signals without applying any non-linearities.

- Generally, splines excel in accurately representing low-dimensional functions but struggle with high-dimensional data due to the curse of dimensionality.
- On the other hand, MLPs are proficient in feature learning but may falter in optimizing univariate functions.
- Hence, **KANs** merge the accuracy of splines with the feature learning capability of MLPs.

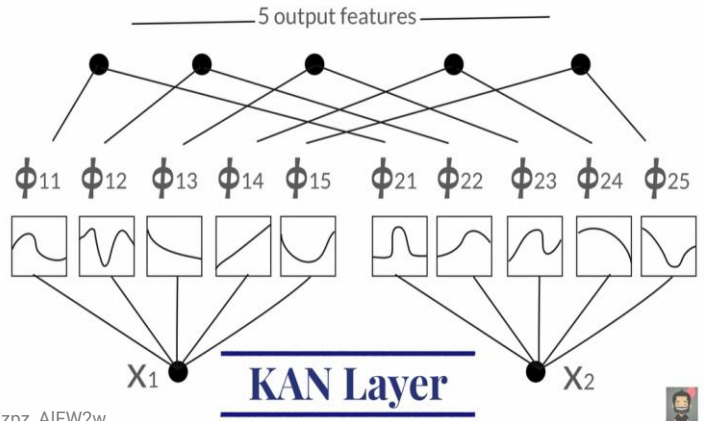
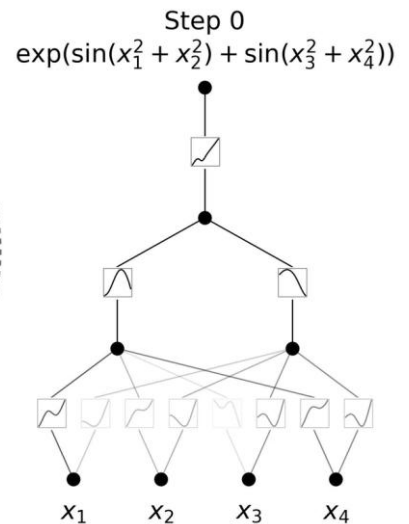
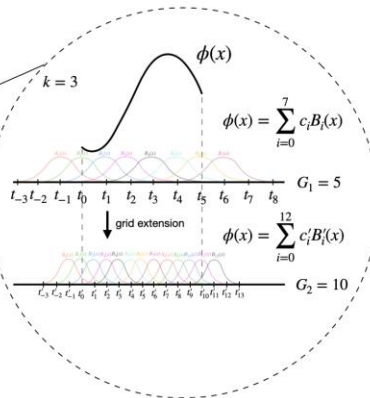
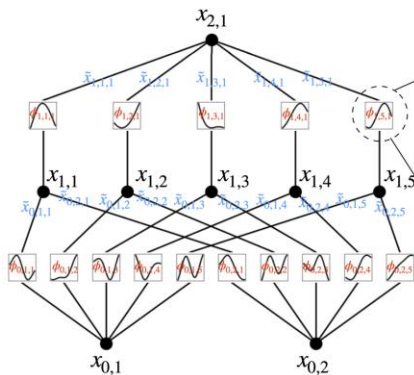


Image source: 11MAY2024 https://www.youtube.com/watch?v=7zpz_AIFW2w

51

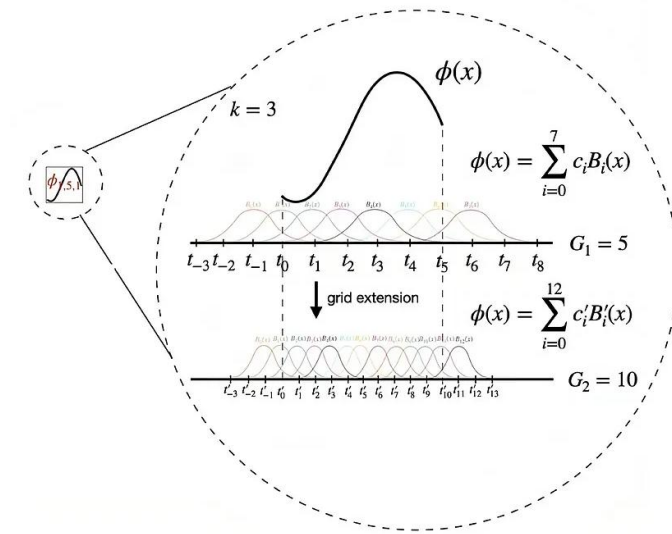
Kolmogorov-Arnold Network (arxiv 2024)



"KAN: Kolmogorov-Arnold Networks," arxiv 30APR2024 <https://arxiv.org/abs/2404.19756> , <https://github.com/KindXiaoming/pykan> , <https://kindxiaoming.github.io/pykan/>

52

A Kolmogorov-Arnold Network (arxiv 2024)



The general formula for a spline in the context of **KANs** can be expressed using **B-splines** as:

$$\text{spline}(x) = \sum_i c_i B_i(x)$$

- $\text{spline}(x)$ represents the spline function.
- c_i are the coefficients that are optimized during training.
- $B_i(x)$ are the **B-spline** basis functions defined over a grid.
- The grid points define the intervals where each basis function B_i is active and significantly affects the shape and smoothness of the spline.

"KAN: Kolmogorov-Arnold Networks," arxiv 30APR2024 <https://arxiv.org/abs/2404.19756>, <https://github.com/KindXiaoming/pykan>, <https://kindxiaoming.github.io/pykan/>

53

A Kolmogorov-Arnold Network (arxiv 2024)

- Pros of **KANs** (according to the **KAN** paper):
 - Interpretability
 - Because learnable results are functions not just a weight matrix for linear equations.
 - Continual learning without catastrophic forgetting
 - MLPs usually use global activations and any local change may propagate uncontrollably to regions far away, destroying the information being stored there.
 - In **KANs**, since spline bases are local, a sample will only affect a few nearby spline coefficients, leaving far-away coefficients intact.
 - Parameter efficient
 - Beating the curse of dimensionality
 - Additional complexity without retraining
 - One can first train a **KAN** with fewer parameters and then extend it to a **KAN** with more parameters by simply making its spline grids finer, without the need to retrain the larger model from scratch.
- Cons of **KANs** (according to the **KAN** paper):
 - Slow training, usually 10x slower than MLPs at the same numbers of params
 - Not GPU-efficient as different activation functions cannot leverage batch computation
 - Lack of testing in real-world ML datasets (e.g., no test for sequential data and large-scale dataset)
 - Potential overfit, especially in scenarios with limited data

Recommended materials:

- Easy overview: (11MAY2024): https://www.youtube.com/watch?v=7zpz_AIFW2w
- More in detail (11MAY2024): <https://www.youtube.com/watch?v=-PF1kkwWdnM>

54

A Kolmogorov-Arnold Network (arxiv 2024)

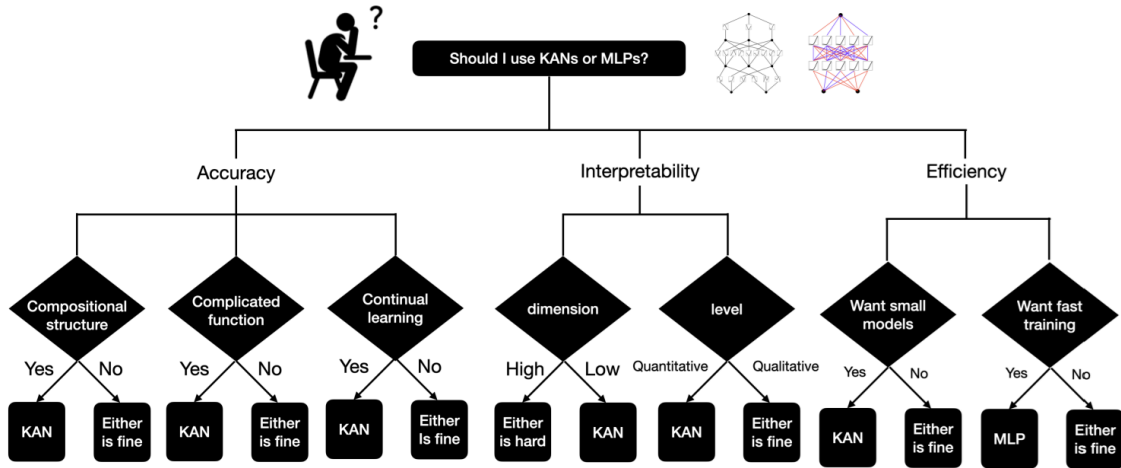


Figure 6.1: Should I use KANs or MLPs?

"KAN: Kolmogorov-Arnold Networks," arxiv 30APR2024 <https://arxiv.org/abs/2404.19756> , <https://github.com/KindXiaoming/pykan> , <https://kindxiaoming.github.io/pykan/>

55



56